

Cryptarithmic Problem

1. Aim

To implement a **Cryptarithmic puzzle** using Artificial Intelligence techniques, where each letter represents a unique digit from 0 to 9, and determine the digit assignment that satisfies the given arithmetic equation.

2. Problem Statement

Cryptarithmic is a mathematical puzzle in which the letters of an arithmetic expression are replaced by different digits. The objective is to find a unique digit for each letter such that the complete arithmetic operation is mathematically correct. No two letters can represent the same digit, and a leading letter cannot represent zero.

3. Solution

The problem can be solved using **constraint satisfaction and backtracking**. Different possible digits are assigned to the letters while maintaining the constraints of the puzzle. Each assignment is checked to determine whether it satisfies the arithmetic equation. Invalid assignments are rejected, and the search continues until a valid solution is found.

For example, the puzzle **SEND + MORE = MONEY** can be solved by assigning different digits to the letters **S, E, N, D, M, O, R, Y**.

4. Algorithm

1. Identify all unique letters in the cryptarithmic equation.
2. Assign a different digit from 0 to 9 to each letter.
3. Ensure that leading letters are not assigned zero.
4. Generate possible digit assignments using backtracking.
5. Convert the letters into their corresponding numerical values.
6. Check whether the arithmetic equation is satisfied.
7. If the equation is not satisfied, backtrack and try another assignment.
8. Continue until a valid assignment is found.
9. Display the digit assigned to each letter and the resulting equation.

5. Program

```
from itertools import permutations
```

```
def solve_cryptarithm():  
    letters = "SENDMORY"
```

```
for digits in permutations(range(10), len(letters)):
```

```
    S, E, N, D, M, O, R, Y = digits
```

```
    # Leading letters cannot be zero
```

```
    if S == 0 or M == 0:
```

```
        continue
```

```
    SEND = 1000*S + 100*E + 10*N + D
```

```
    MORE = 1000*M + 100*O + 10*R + E
```

```
    MONEY = 10000*M + 1000*O + 100*N + 10*E  
+ Y
```

```
    if SEND + MORE == MONEY:
```

```
        print("Solution Found:")
```

```
        print("S =", S)
```

```
        print("E =", E)
```

```
        print("N =", N)
```

```
        print("D =", D)
```

```
        print("M =", M)
```

```
        print("O =", O)
```

```
print("R =", R)
```

```
print("Y =", Y)
```

```
print("\nEquation:")
```

```
print(SEND, "+", MORE, "=", MONEY)
```

```
return
```

```
solve_cryptarithm()
```

6. Result

The Cryptarithmic problem was successfully implemented using **backtracking and constraint satisfaction**. The program searches through possible digit assignments and identifies a valid assignment where every letter represents a unique digit.

Output:

Solution Found:

S = 9

E = 5

N = 6

D = 7

M = 1

O = 0

$$R = 8$$

$$Y = 2$$

Equation:

$$9567 + 1085 = 10652$$

Thus, the cryptarithmic puzzle **SEND + MORE = MONEY** was successfully solved, satisfying all the given constraints.

The screenshot shows a Python IDE with a project named 'PythonProject10'. The file explorer on the left lists various Python files, including 'Cript-Arithmetic.py'. The main editor displays the code for 'Cript-Arithmetic.py', which defines the variables S, E, N, D, M, O, R, Y, and P, and implements a brute-force search algorithm to solve the cryptarithmic puzzle. The code includes a loop that iterates through possible values for S, E, N, D, M, O, R, and Y, and checks if the equation SEND + MORE = MONEY is satisfied. The output of the program is shown in the Run console, displaying the solution: SEND = 9567, MORE = 1085, and MONEY = 10652. The process finished with exit code 0.

```
PythonProject10
├── .venv
│   ├── library
│   └── root
├── 8-Puzzle.py
├── 8-Queen.py
├── ALPHA BETA PRUNING.py
├── BFS.py
├── Cannibal.py
├── Cript-Arithmetic.py
├── DECISION TREE.py
├── DFS.py
├── FEED FORWARD NEURAL NETWORK.py
├── MAP COLOURING.py
├── MIN_COST.py
└── MINIMAX.py
```

```
Travelling Salesman.py  MIN_COST.py  Cannibal.py  Vacuum Cleaner.py  Water Jug.py  Cript-Arithmetic.py  8-Puzzle.py
```

```
1  S, E, N, D, M, O, R, Y = P
2
3  if S == 0 or M == 0:
4      continue
5
6  send = 1000*S + 100*E + 10*N + D
7  more = 1000*M + 100*O + 10*R + E
8  money = 10000*M + 1000*O + 100*N + 10*E + Y
9
10 if send + more == money:
11     print("SEND =", send)
12     print("MORE =", more)
13     print("MONEY =", money)
14     break
```

```
Run: Cript-Arithmetic
C:\Users\r290\PycharmProjects\PythonProject10\.venv\scripts\python.exe C:\Users\r290\PycharmProjects\PythonProject10\Cript-Arithmetic.py
SEND = 9567
MORE = 1085
MONEY = 10652
Process finished with exit code 0
```